

AI-DJ: Spotify Hit Predictor

Presented by : Antoine Paulis
Charlotte Michon
Mohamed-Khalil Ankri

Presentation Outline

- I. Milestone 1 + improvement**
- II. Code testing**
- III. Model deployment**
- IV. Model pipeline**
- V. Model serving**



Milestone 1 + improvement

Milestone 1 + improvement

- Milestone 1 recap:
 - **Purpose** : Tool to predict if a song will be a “hit” or a “flop”, by training on a Spotify dataset (*kaggle*) of different genres of music.
 - **ML model** : random forest for a binary classification problem.
 - **MLOps Automation** : Gitflow on Github and Google Cloud.
- Google Cloud improvement:
 - ~~Bucket~~ => **BigQuery** : tabular data + better querrying and scalability.
- Training folder improvement:

Milestone 1

- 📁 **Training**
 - EDA.ipynb
 - evaluate.py
 - gb.py
 - knn.py
 - rf.py

Milestone 2

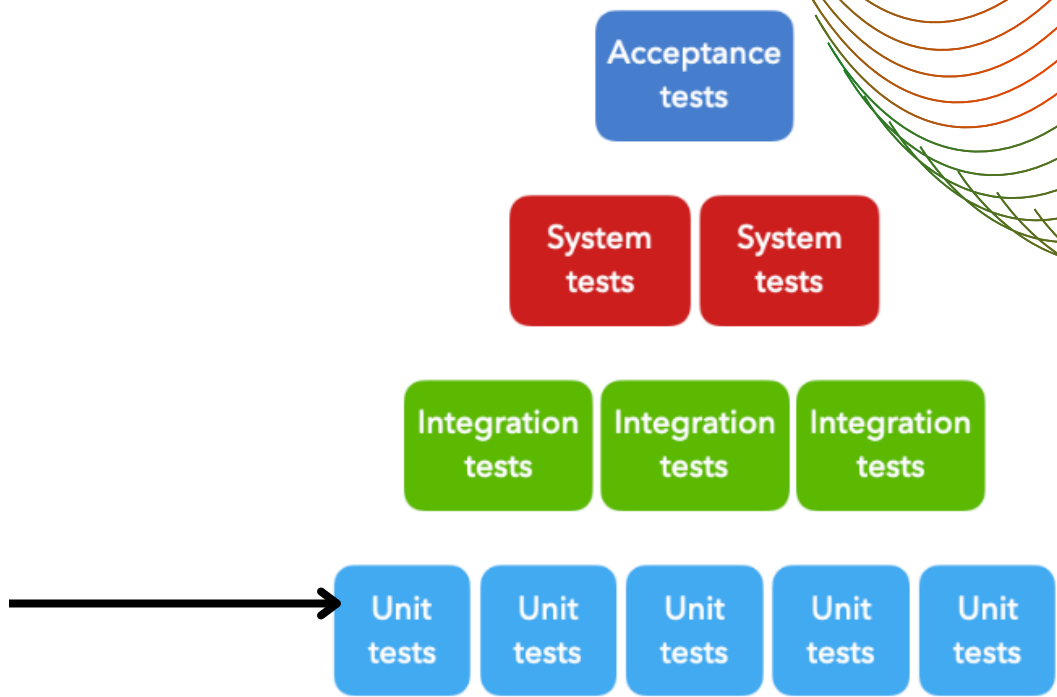
- 📁 **Training**
 - EDA.ipynb
 - evaluate.py
 - gb.py
 - importDataset.py
 - knn.py
 - rf.py
 - train.py
 - trainingPreprocess.py

Code testing

Code testing

- **PyTests**: catch bug early, improve code quality and documentation.
 - **Unit tests** : components that have single responsibilities.
 - `test_evaluate.py` : tests the `metricsEval(yValFold, yPred)`.

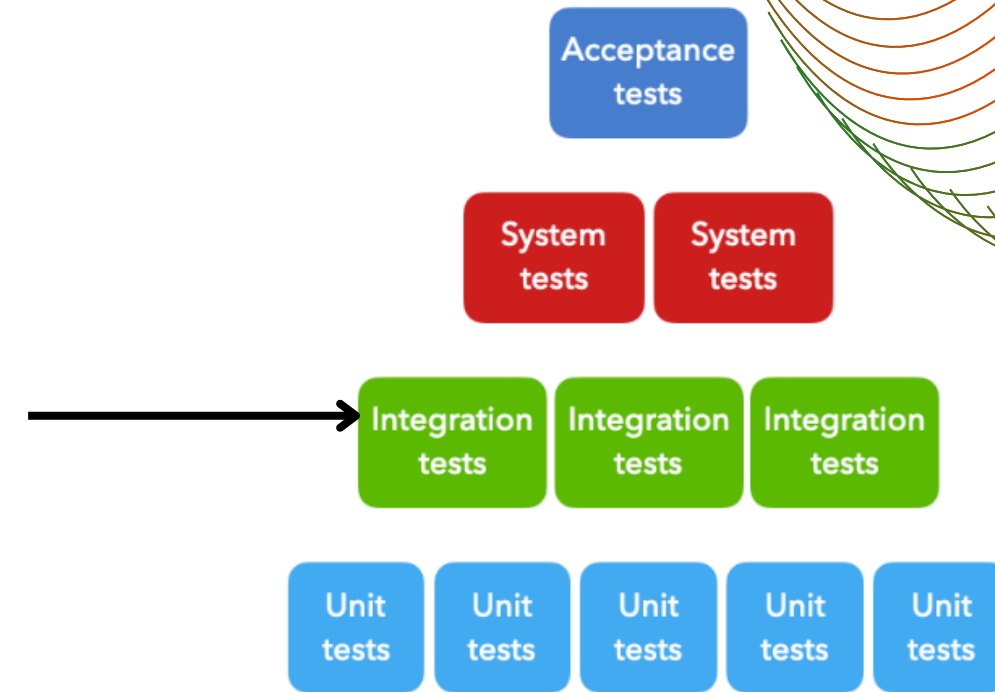
Actual class	Predicted class		Total
	Positive	Negative	
Positive	True Positive	False Negative	P
Negative	False Positive	True Negative	N



```
def test_metricsEval():
    yValFold = [1, 0, 1, 1, 0]
    yPred = [1, 0, 0, 1, 1]
    TP = 2
    FN = 1
    P = TP + FN
    TN = 1
    FP = 1
    N = TN + FP
    accuracy = (TP+TN)/(P+N)
    recall = TP / (TP + FN)
    precision = TP / (TP + FP)
    f1 = 2*(precision*recall)/(precision+recall)
    accuracyFct, recallFct, f1Fct, precisionFct = metricsEval(yValFold, yPred)
    assert accuracyFct == accuracy
    assert recall == recallFct
    assert precision == precisionFct
    assert f1 == f1Fct
```


Code testing

- **PyTests:**
 - **Integration tests :** integration of individual components.
 - **test_dataset_import.py :**
It tests the `importBigQuery()` by checking if the imported dataset contains the corresponding column.
 - **test_trainingPreprocess.py :**
It tests the `trainingPreprocess()` by checking if the splitted X and y contain the corresponding column.



```
def test_dataset_import():
    df = importBigQuery()
    columns = ["track_id", "artists", "album_name", "track_name", "popularity", "duration_ms", "explicit", "danceability",
               "energy", "key", "loudness", "mode", "speechiness", "acousticness", "instrumentalness", "liveness", "valence",
               "tempo", "time_signature", "track_genre"]
    for column in columns:
        assert column in df.columns
```

```
def test_trainingPreprocess():
    df = importBigQuery()
    X,y = trainingPreprocess(df)
    XColumns = ["duration_ms", "danceability", "energy", "key", "loudness", "mode",
               "speechiness", "acousticness", "instrumentalness", "liveness", "valence", "tempo",
               "time_signature"]
    for column in XColumns:
        assert column in X.columns
    assert "hit" in y.name
```

API layer deployment

API layer build

1. REpresentational State Transfer API build

- **FastAPI:**
 - modern framework, can generate auto-documentation and built-in-swagger (based on OpenAI specification) for easy testing of API endpoints
 - asynchronous support, real-time performance and low-latency architecture;
 - automatic input validation, uses Pydantic to validate the Spotify data schema
- Run locally, in the Docker file container and deployed on Google Cloud Storage
- Endpoints: get “health status” / past predictions, post method for predictions



2. Containerisation with uv

- Management with uv for fast compilation and dependencies management
- Dockerfile build based on Python 3.12 slim (for compatibility and stability)
- Modern approach with pyproject.toml (and uv.lock) for automatic management

3. Cloud deployment

- Google Cloud Run, on europe-west4: <https://ai-dj-api-343602206157.europe-west4.run.app/health>
- Adaptations required for local version to be deployed

REST API

1. REST API

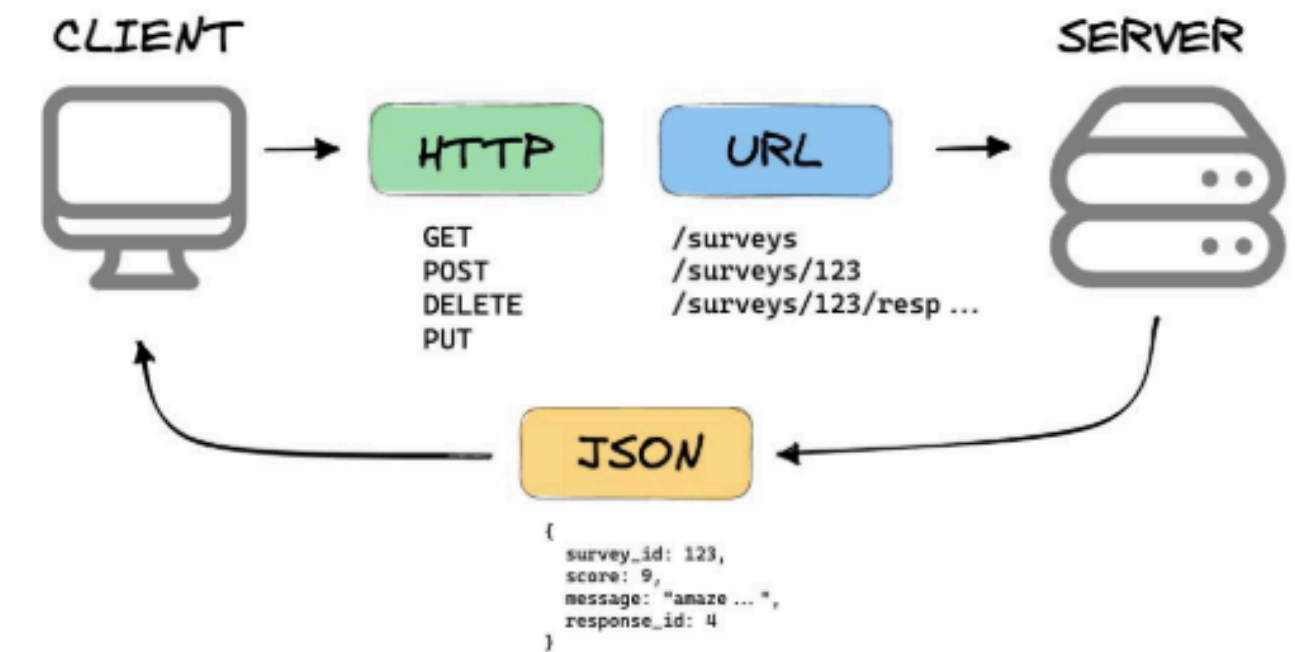
- GET methods : <https://ai-dj-api-343602206157.europe-west4.run.app/>
 - `past_predictions`
 - `past_predictions/1` (by id)

```
Body Cookies Headers (6) Test Results
{} JSON Preview Visualize
1 [
2   {
3     "id": 0,
4     "query": "MOCK_FLOP",
5     "prediction": "Flop",
6     "confidence": "N/A"
7   },
8   {
9     "id": 1,
10    "source": "manual_input",
11    "prediction": "Flop",
12    "class_index": 0,
13    "features_received": {
14      "danceability": 1,
15      "energy": 1,
16      "key": 5,
17      "loudness": -5.0,
18      "mode": 1,
19      "speechiness": 1,
20      "acousticness": 1,
21      "instrumentalness": 0.01,
22      "liveness": 1,
23      "valence": 1,
24      "tempo": 120,
25      "duration_ms": 210000,
```

```
Body Cookies Headers (6) Test Results
{} JSON Preview Visualize
1 {
2   "id": 1,
3   "source": "manual_input",
4   "prediction": "Flop",
5   "class_index": 0,
6   "features_received": {
7     "danceability": 1,
8     "energy": 1,
9     "key": 5,
10    "loudness": -5.0,
11    "mode": 1,
12    "speechiness": 1,
13    "acousticness": 1,
14    "instrumentalness": 0.01,
15    "liveness": 1,
16    "valence": 1,
17    "tempo": 120,
18    "duration_ms": 210000,
19    "time_signature": 4
20  }
21 }
```

REpresentational State Transfer (REST) API

Roy Fielding's PhD thesis: [Architectural Styles and the Design of Network-based Software Architectures](https://www.royfielding.com/thesis/)



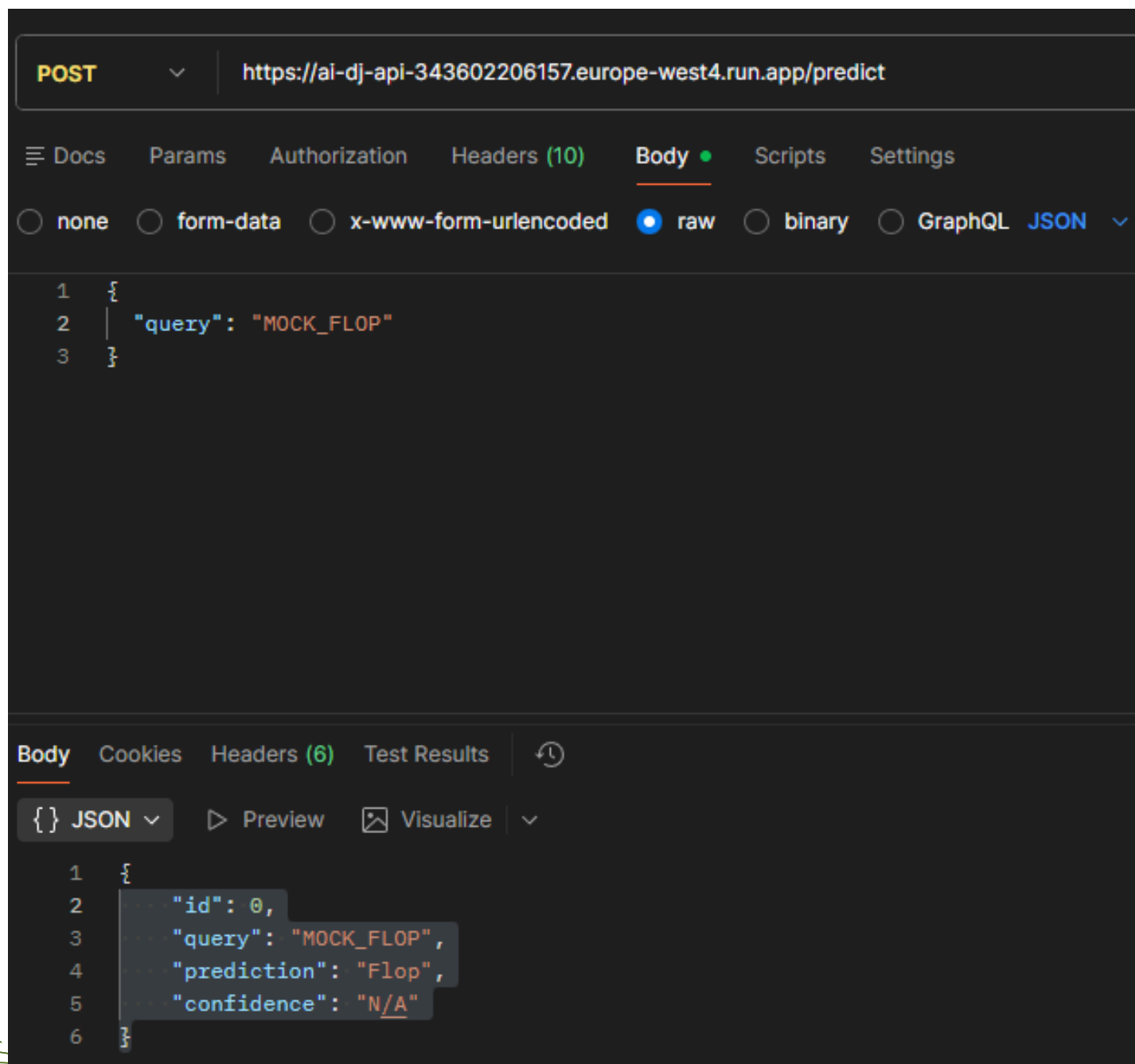
<https://mannhowie.com/rest-api>

```
43 @app.get("/past_predictions")
44 def get_history():
45     return history
46
47
48 @app.get("/past_predictions/{prediction_id}")
49 def get_prediction_by_id(prediction_id: int):
50
51     if prediction_id < 0 or prediction_id >= len(history):
52         raise HTTPException(status_code=404, detail="Prediction ID not found")
53
54     return history[prediction_id]
```

REST API: POST method

1. REST API

- POST methods : <https://ai-dj-api-343602206157.europe-west4.run.app/>
 - predict: asynchronous function, query format with a title, prediction features selected (hit or not, confidence)
 - predict_manual: asynchronous function, use dictionary of features



```
16 @app.post("/predict")
17 async def predict(data: dict):
18     query = data.get("query")
19     if not query:
20         raise HTTPException(status_code=400, detail="Query missing")
21
22     # 1. Fetch from Spotify
23     audio_features = spotify_service.get_features(query)
24     if not audio_features:
25         raise HTTPException(status_code=404, detail="Track not found")
26
27     # 2. Get Prediction from Vertex AI
28     try:
29         result = prediction_service.predict(audio_features)
30     except Exception as e:
31         raise HTTPException(status_code=500, detail=f"Vertex AI Error: {str(e)}")
32
33     # 3. Format & Store
34     response = {
35         "id": len(history),
36         "query": query,
37         "prediction": result["class_name"],
38         "confidence": result.get("confidence", "N/A")
39     }
40     history.append(response)
41     return response
```

REST API: POST method

- **Future work:**

- **re-implement a fancy landing homepage,**

```
@app.get("/")
```

```
def home():
```

```
    return {"Homepage of Spotify Hit Predictor"}
```

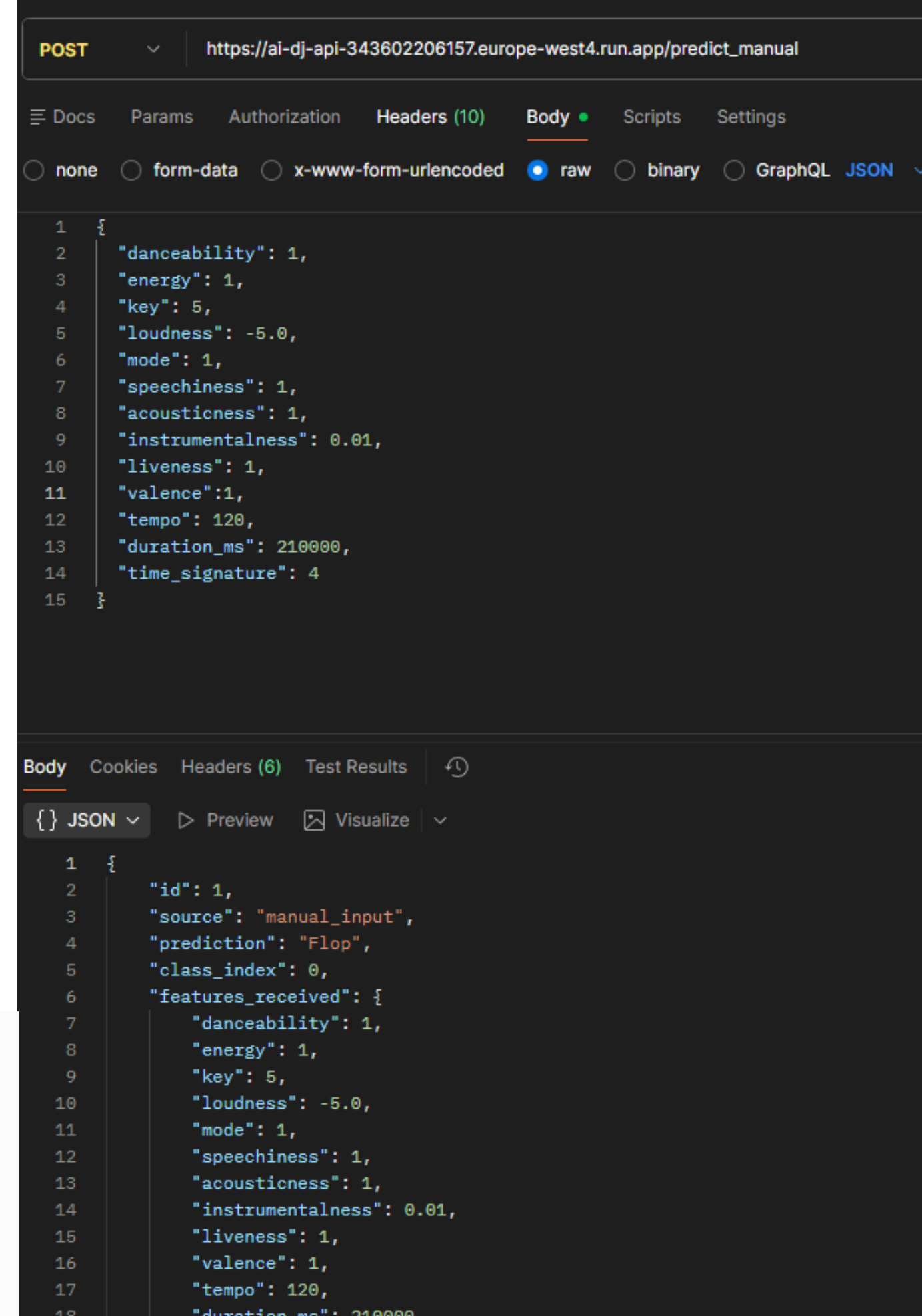
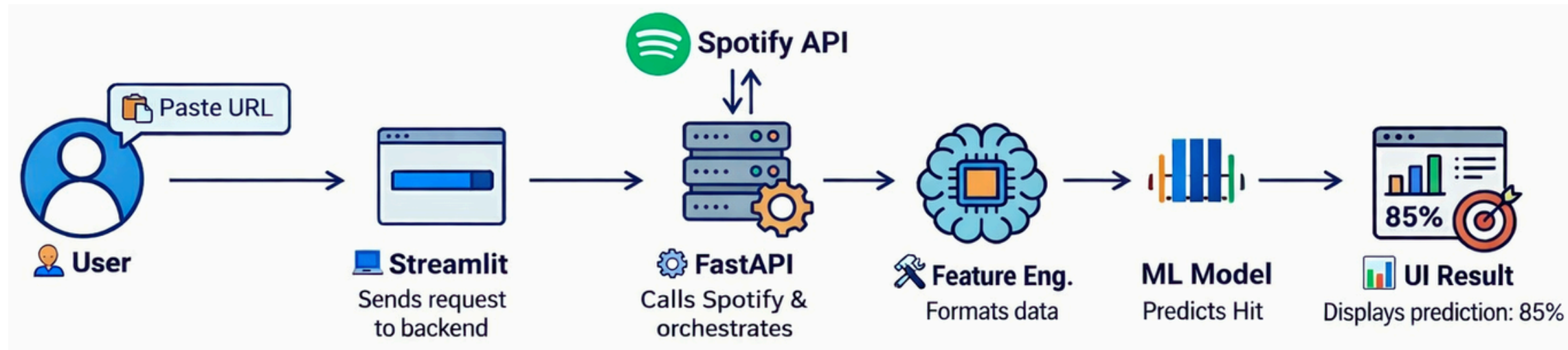
- **a top-5-features retrieving page (or simple integration in POST predict)**

```
@app.get("/features")
```

```
def get_top_features():
```

```
    return {"top_5": list_top5}
```

- **management of url for GET method: submission via Streamlit interface**



Model deployment

2. Containerisation with uv

- fast compilations: while deploying, required to check the dependencies to manage versions with .toml
- Python 3.12 (vs 3.13) for Vertex AI
- uv sync. during deployment
- port variable for GCS deployment

```
pyproject.toml
1 [project]
2 name = "ai-dj"
3 version = "0.1.0"
4 description = "Dependencies for AI-DJ Project"
5 readme = "README.md"
6 requires-python = ">=3.12,<3.14"
7
8 dependencies = [
9     "fastapi>=0.115.0",
10    "uvicorn[standard]>=0.30.0",
11    "spotipy>=2.24.0",
12    "pandas>=2.2.0",
13    "scikit-learn>=1.5.0",
14    "numpy>=2.0.0",
15    "python-dotenv>=1.0.0",
16    "redis>=5.0.0",
17    "google-cloud-bigquery>=3.40.1",
18    "db-dtypes>=1.5.0",
19    "google-cloud-storage>=3.10.1",
20    "google-cloud-aiplatform>=1.143.0",
21    "kfp>=2.16.0",
22 ]
```

```
Dockerfile > ...
1 # Use 3.12 for maximum stability with Vertex AI libraries
2 FROM python:3.12-slim
3
4 # Install UV
5 COPY --from=ghcr.io/astral-sh/uv:latest /uv /usr/local/bin/uv
6
7 # Set working directory
8 WORKDIR /app
9
10 # Copy dependency files first to cache the 'install' layer
11 COPY pyproject.toml .
12 # If you have a requirements.txt instead, use: COPY requirements.txt .
13
14 # Install dependencies using uv into the system python
15 RUN uv pip install --system --no-cache fastapi uvicorn google-cloud-aiplatform spotipy python-dotenv pandas
16
17 # Copy the rest of the source code
18 COPY . .
19
20 # Cloud Run uses the PORT env variable
21 EXPOSE 8080
22
23 # The CMD needs to point to the file where 'app = FastAPI()' lives
24 # use the API path we built :
25 CMD ["sh", "-c", "uvicorn src.api.main:app --host 0.0.0.0 --port ${PORT:-8080}"]
```


Model deployment

3. Cloud deployment

- from the container: needed to fetch models
 - used bucket storage for .pkl file to download
 - IAM configurations (made during the lab)
- settings to access the data

```
src/ui/app.py
23 # Specify absolute path for Docker image
24 BASE_DIR = os.path.dirname(os.path.abspath(__file__))
25 + BUCKET_NAME = "ai-dj-487610-bucket"
26 + MODEL_DIR = os.path.join(BASE_DIR, "../training")
27 +
28 + def download_models():
29 +     client = storage.Client(project="ai-dj-487610")
30 +     bucket = client.bucket(BUCKET_NAME)
31 +     for fname in ["model.pkl", "columns.pkl", "top_5.pkl"]:
32 +         dest = os.path.join(MODEL_DIR, fname)
33 +         if not os.path.exists(dest):
34 +             print(f"Downloading {fname} from GCS...")
35 +             bucket.blob(f"pkl/{fname}").download_to_filename(dest)
36 +             print(f"{fname} downloaded.")
37 +
38 + download_models()
39
40 model = pickle.load(open(os.path.join(BASE_DIR, "../training/model.pkl"), "rb"))
41 model_columns = pickle.load(open(os.path.join(BASE_DIR, "../training/columns.pkl"), "rb"))
42 list_top5 = pickle.load(open(os.path.join(BASE_DIR, "../training/top_5.pkl"), "rb"))
43
44 classes = ["Not a Hit", "Hit"]
45 history = []
46
47 def fetch_features_from_spotify(track_input: str):
48     if "open.spotify.com" in track_input or "track" in track_input:
49         # ID extraction from link
50         track_id = track_input.split("/")[-1].split("?")[0]
```

```
docs > uv-config.txt
1 # Pip compatibility mode, with our requirements.txt
2 # create a new Python virtual environment
3 uv venv --python 3.12
4
5 # UV configuration
6 .venv\Scripts\activate # Windows
7
8 # install from requirements.txt
9 uv pip install -r requirements.txt
10
11 uv run uvicorn app:app --host 0.0.0.0 --port 8080 --reload
12
13
14 docker build -t ui-app .
15 docker run -p 8080:8080 ui-app
16
17 # Deployment successful
18 (AI-Dj) PS C:\Users\huiyi\Documents\ULiege\Master\Master-m1_local\AI-Dj> gcloud run deploy ui-app --project ai-dj-487610
19 Building using Dockerfile and deploying container to Cloud Run service [ui-app] in project [ai-dj-487610]
20 OK Building and deploying... Done.
21 OK Validating configuration...
22 OK Uploading sources...
23 OK Building Container... Logs are available at [https://console.cloud.google.com/cloud-build/builds;regi]
24 OK Creating Revision...
25 OK Routing traffic...
26 OK Setting IAM Policy...
27 Done.
28 Service [ui-app] revision [ui-app-00003-zq5] has been deployed and is serving 100 percent of traffic.
29 Service URL: https://ui-app-343602206157.europe-west1.run.app
```

→ Automatisation of Training and Deployment did not require to download model

→ Endpoints are directly connected to the pipeline

Next stage: User interface to show our results...

and deploy it on GCS :)

Model pipeline

Transitioning to Orchestrated ML: From Scripts to DAGs

1. Orchestration

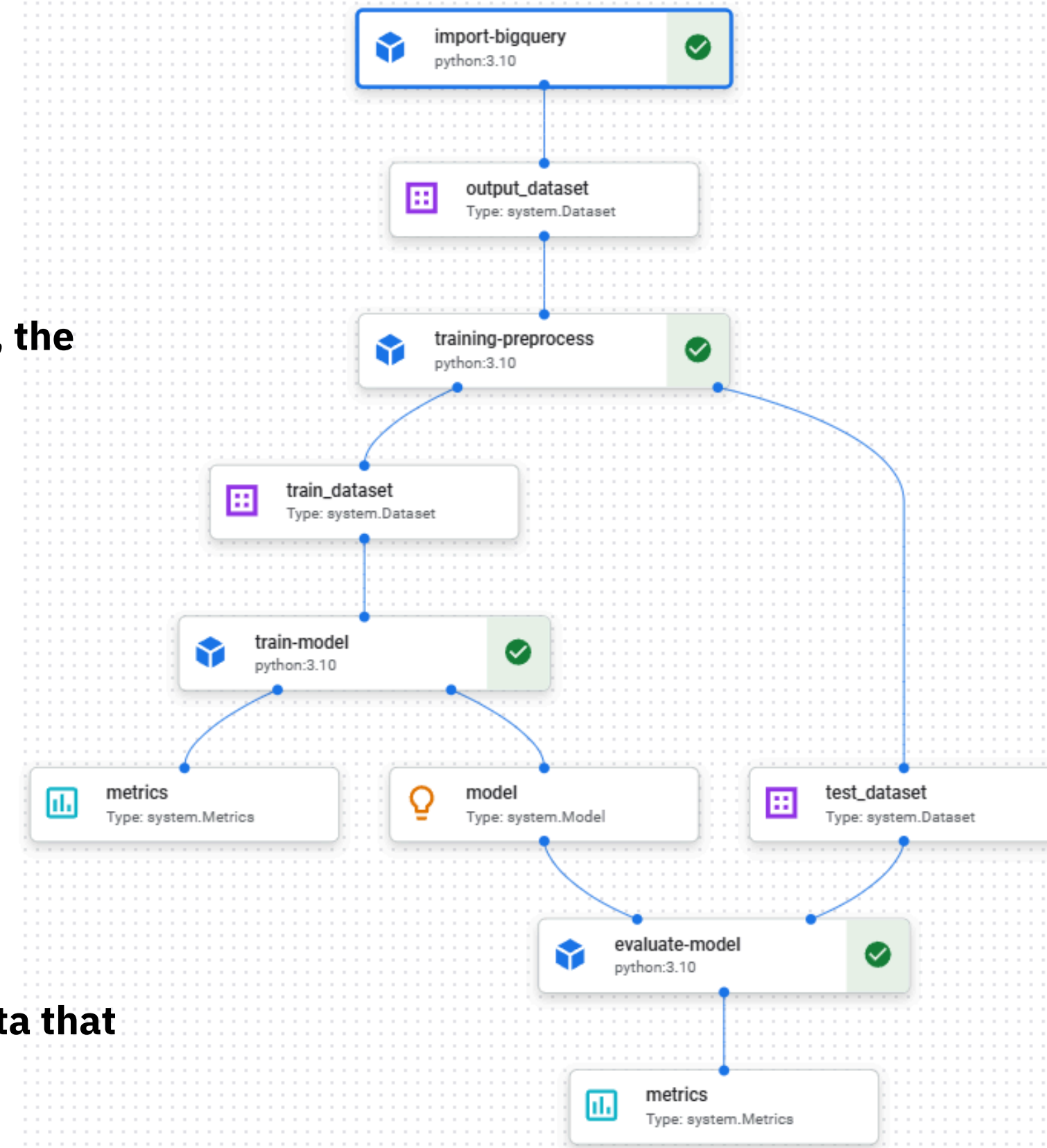
- **Orchestration vs. Execution:** We moved from a manual, linear script to a Directed Acyclic Graph (DAG).
- **System Reliability:** The pipeline is now State-Aware. If a single step fails, the entire process halts safely.

2. Containerized Isolation

- Each box in the graph is a Standalone Docker Container.
- This eliminates the "Works on my machine" problem by freezing the environment in the cloud.

3. Automatic Data Lineage

- Vertex AI tracks every Artifact (the Dataset, the Model, the Metrics).
- We can now trace any model back to the exact version of the BigQuery data that created it.



The Artifact Handshake & Managed Metadata

1. From Session Data → Persistent Data

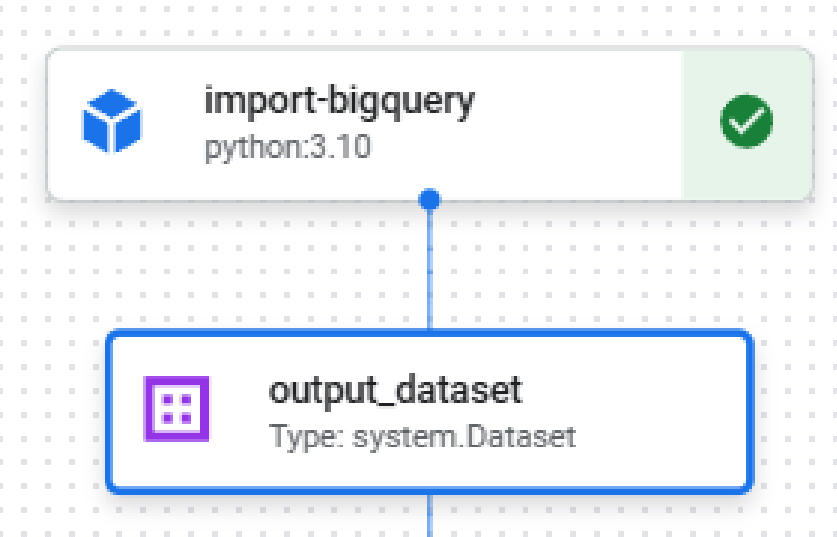
- Switched from in-memory data to stored artifacts
- Each pipeline run creates a timestamped dataset version for reproducibility

2. Metadata & Auditability

- Log key metadata (query, schema, row counts)
- Full traceability from data source to trained model in Vertex AI

3. Reduced Data Drift

- Training uses fixed dataset snapshots
- Ensures consistent evaluation and reproducibility



Artifact Info

[Open in ML Metadata](#)

Name	output_dataset
Type	system.Dataset
URI	gs://ai-dj-487610-bucket/pipeline_root/343602206157/ai-dj-training-pipeline-20260330171442/import-bigquery_-720587210373464064/output_dataset

Properties

All properties of the artifact.

row_count	114000
source	BigQuery

The Training Component

- **Containerized Isolation (Decorator)** : Each step runs inside its own container with all required libraries.
- **Artifact Handshake (Input)** : Components don't use fixed file paths anymore.
- **Experiment Tracking (Metrics)** : Model performance (like accuracy) is logged, not just printed.
- **Data Lineage & Metadata (Output)** : We save more than just the model , we attach useful info (e.g., important features).



	model
	Type: system.Model
URI	gs://ai-dj-487610-bucket/pipeline_root/343602206157/ai-dj-training-pipeline-20260330202727/train-model_4280686564100538368/model
Created	March 30, 2026 8:35 PM
Last Modified	March 30, 2026 8:37 PM
top_5_features	["acousticness", "energy", "loudness", "valence", "instrumentalness"]
framework	scikit-learn
columns	["duration_ms", "danceability", "energy", "key", "loudness", "mode", "speechiness", ...
Pipeline runs	ai-dj-training-pipeline-20260330202727

	metrics
	Type: system.Metrics
URI	gs://ai-dj-487610-bucket/pipeline_root/343602206157/ai-dj-training-pipeline-20260330202727/train-model_4280686564100538368/metrics
Created	March 30, 2026 8:35 PM
Last Modified	March 30, 2026 8:37 PM
training_accuracy	0.9909100877192982
n_estimators	100
Pipeline runs	ai-dj-training-pipeline-20260330202727

Automated Evaluation

Independent Artifact Validation : The evaluation step loads the actual saved model file, not a temporary variable.

Multi-Dimensional Performance Profiling : We evaluate using multiple metrics: accuracy, precision, recall, and F1-score.

Permanent Metadata Archiving : All metrics are stored in Vertex AI’s metadata system.

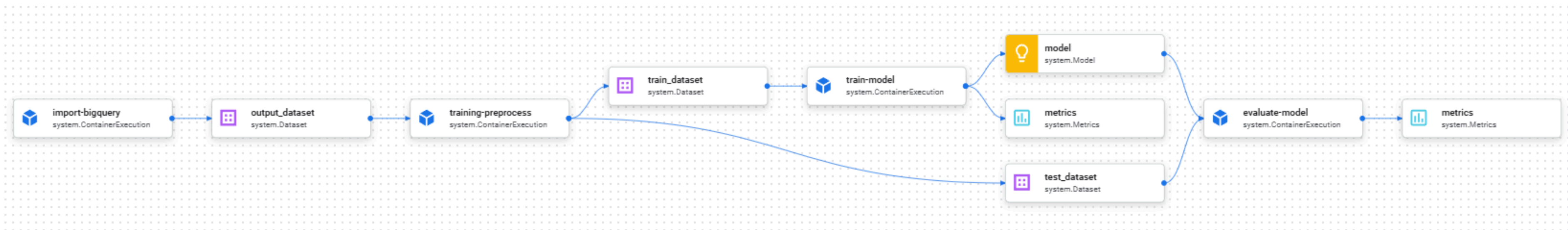


metrics
Type: system.Metrics

URI

gs://ai-dj-487610-bucket/pipeline_root/343602206157/ai-dj-training-pipeline-20260330202727/evaluate-model_6586529573314232320/metrics

Created	March 30, 2026 8:37 PM
Last Modified	March 30, 2026 8:38 PM
precision	0.8332052267486548
accuracy	0.9549122807017544
f1_score	0.6783479349186483
recall	0.5720316622691293
Pipeline runs	ai-dj-training-pipeline-20260330202727



Model serving

Model Registration & Deployment

Model Registration : Uploaded the trained model to Vertex AI → creates a versioned source of truth to track updates.

Containerization : Used a pre-built Scikit-learn container → ensures the cloud environment matches training (no version issues).

Online Deployment : Deployed the model to a Vertex AI endpoint (n1-standard-2) → turns the model into a live API.

Result : The model is now a scalable web service accessible via a URI in europe-west4.

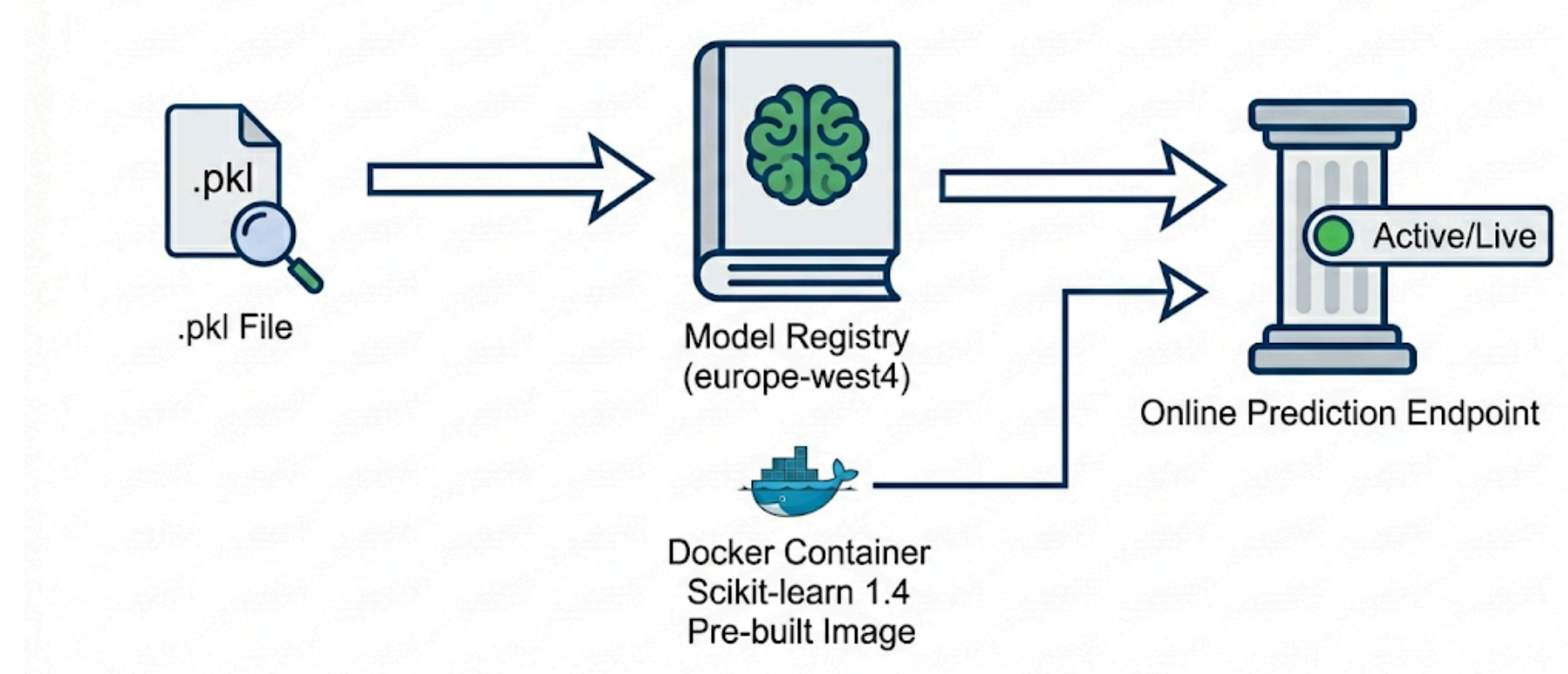
ML Serving Infrastructure Specifications

Hardware Details					
					
Type: n1-standard-2	vCPUs: 2	Memory: 7.5 GB	GPU: None (CPU Only)	Location: europe-west4	Storage: Standard PD

Rationale

Why this matters:

Our n1-standard-2 selection balances cost and performance. A Random Forest model requires standard CPU power, but its 7.5GB of RAM ensures the complete decision tree remains in memory for sub-100ms prediction latency.

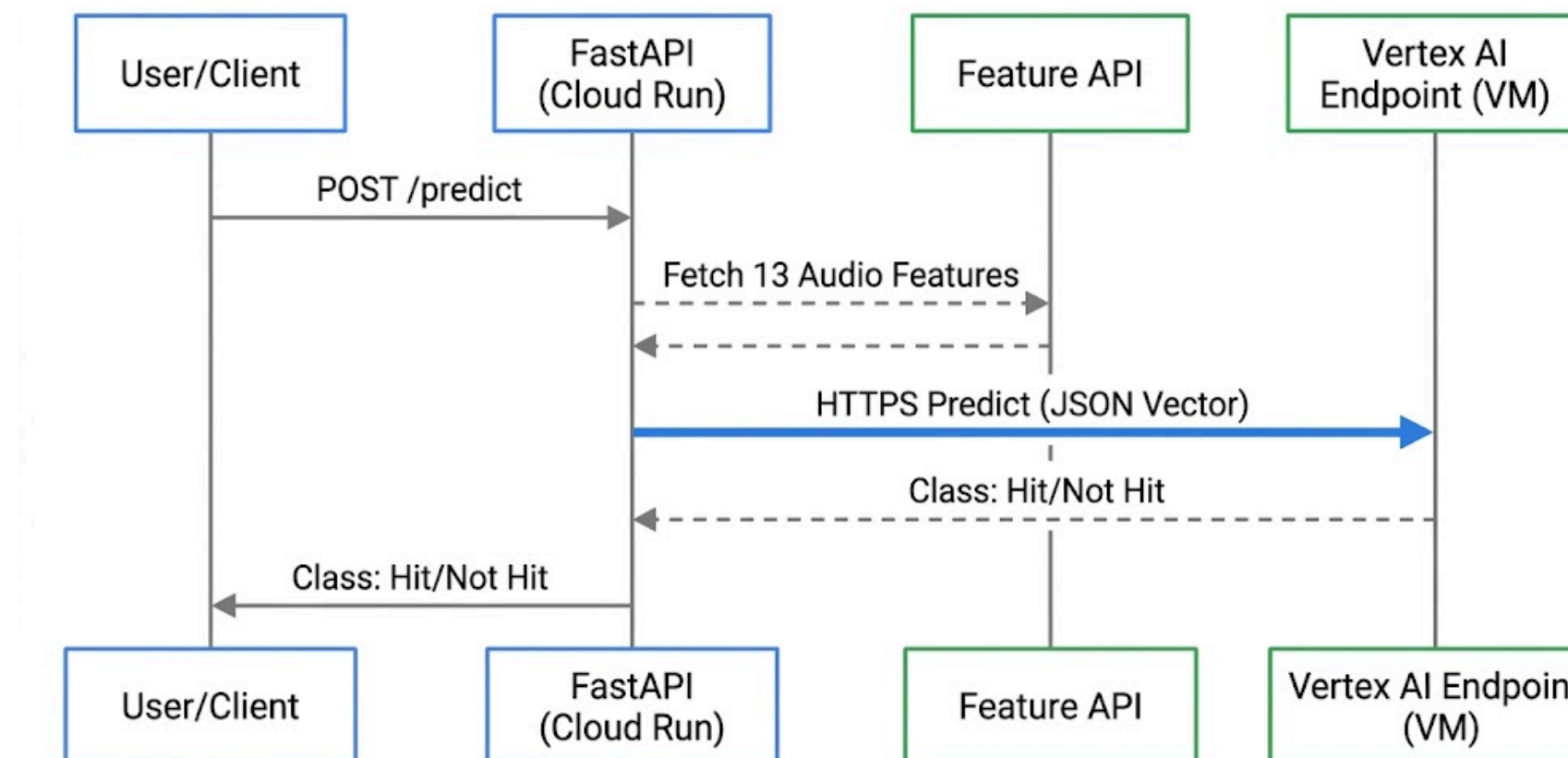
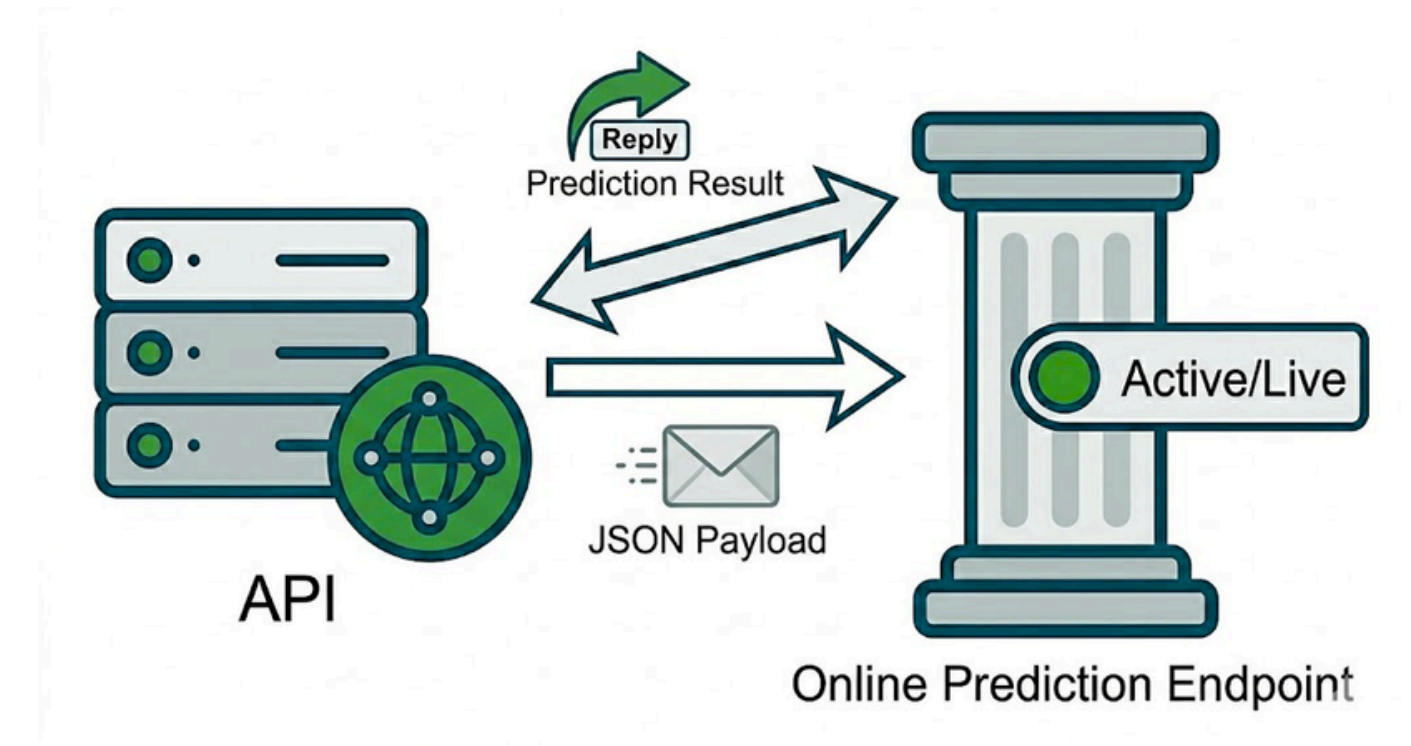


Model Retrieval & Inference Logic

Managed Endpoint: The API calls a live Vertex AI endpoint instead of using a local model.

Feature Alignment: It formats input data into a fixed 13-feature vector to match the model's requirements.

Serverless Deployment: Runs on Cloud Run, scales automatically, and sends secure HTTPS requests to get predictions.





Deployment Improvements

API Layer & Model Retrieval

- **Model-as-a-Service (MaaS):** The API does not load the model locally; it maintains a secure, persistent connection to the Vertex AI Endpoint.
- **The Translator Role:** Ingests raw track data (ID or URL). Normalizes and vectorizes inputs into the exact the features array required by the model.
- **Secure Retrieval:** Communicates with the n1-standard-2 VM via the Google Cloud AI Platform SDK using service-to-service authentication.

```
class PredictionService:
    def __init__(self):
        # Connect to Live Google Cloud Endpoint
        aiplatform.init(project=Config.PROJECT_ID, location=Config.LOCATION)
        self.endpoint = aiplatform.Endpoint(Config.ENDPOINT_ID)

    def predict(self, features: dict):
        # 1. Extract features in the EXACT order the model was trained on
        try:
            input_data = [float(features.get(col, 0)) for col in Config.MODEL_COLUMNS]

            # 2. Call Vertex AI Endpoint
            prediction = self.endpoint.predict(instances=[input_data])

            # 3. Parse the result (Vertex returns floats, we want the class index)
            pred_index = int(prediction.predictions[0])

            return {
                "class_index": pred_index,
                "class_name": Config.CLASSES[pred_index],
                "raw_prediction": prediction.predictions[0]
            }
        except Exception as e:
            print(f"Vertex AI Prediction Error: {e}")
            raise e
```

Name	Default version	Deployment status ↑	Description	Type	Source	Updated
spotify-hit-predictor	1	✔ Deployed	—	Ⓜ Imported	Custom training	Mar 31, 2026, 3:04:33 PM

Cloud Deployment & Infrastructure

- **Containerization:** Packaged the FastAPI application and all dependencies into a Docker Image for environment consistency .
- **Google Cloud Run:** Serverless Execution , Scales automatically from zero to multiple instances based on request volume.
 - **Cost Efficiency:** Resources are only consumed during active prediction requests.
- **Identity & Access Management (IAM):** Configured a dedicated Service Account with Vertex AI User permissions to allow secure, keyless communication between Cloud Run and Vertex AI.
- **Production Endpoint:** Deployed a live, secured HTTPS gateway:
 - <https://ai-dj-api-343602206157.europe-west4.run.app>

```
FROM python:3.12-slim (last pushed 1 week ago)

# Install UV
COPY --from=ghcr.io/astral-sh/uv:latest /uv /usr/local/bin/uv

# Set working directory
WORKDIR /app

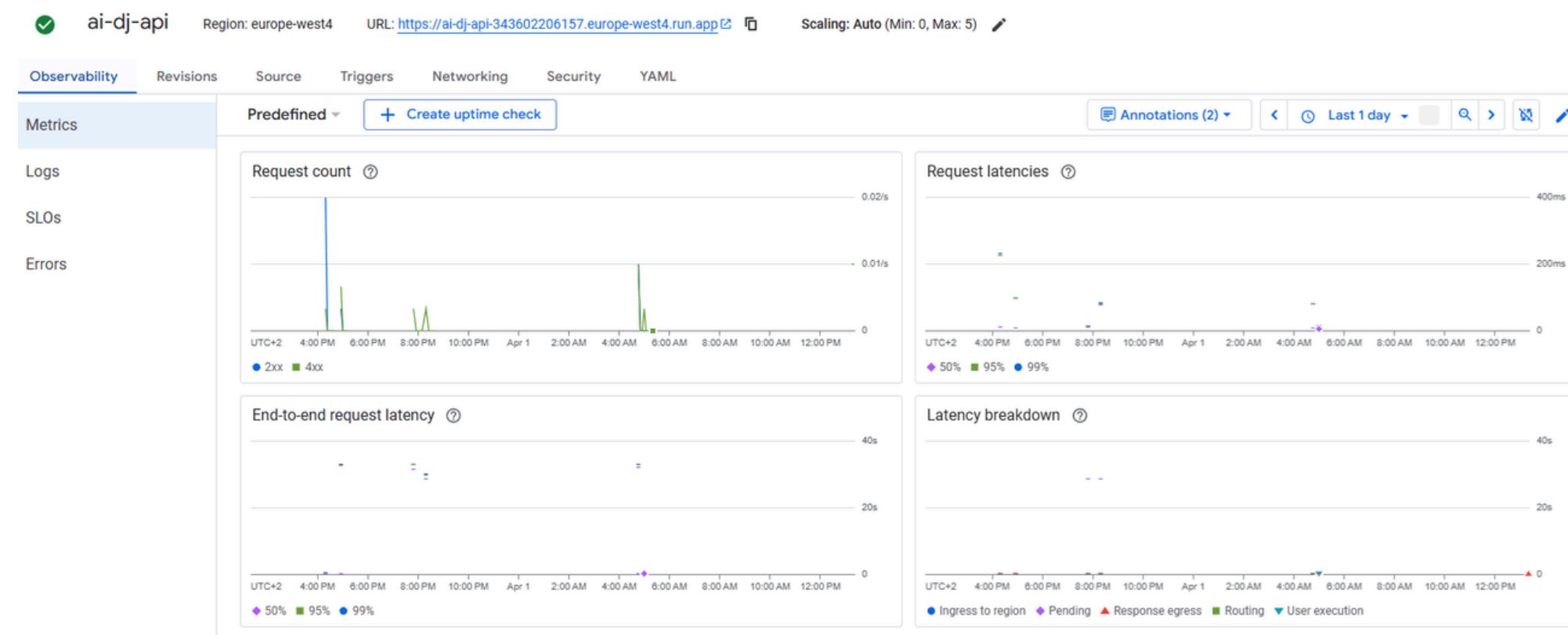
# Copy dependency files first to cache the 'install' layer
COPY pyproject.toml .
# If you have a requirements.txt instead, use: COPY requirements.txt .

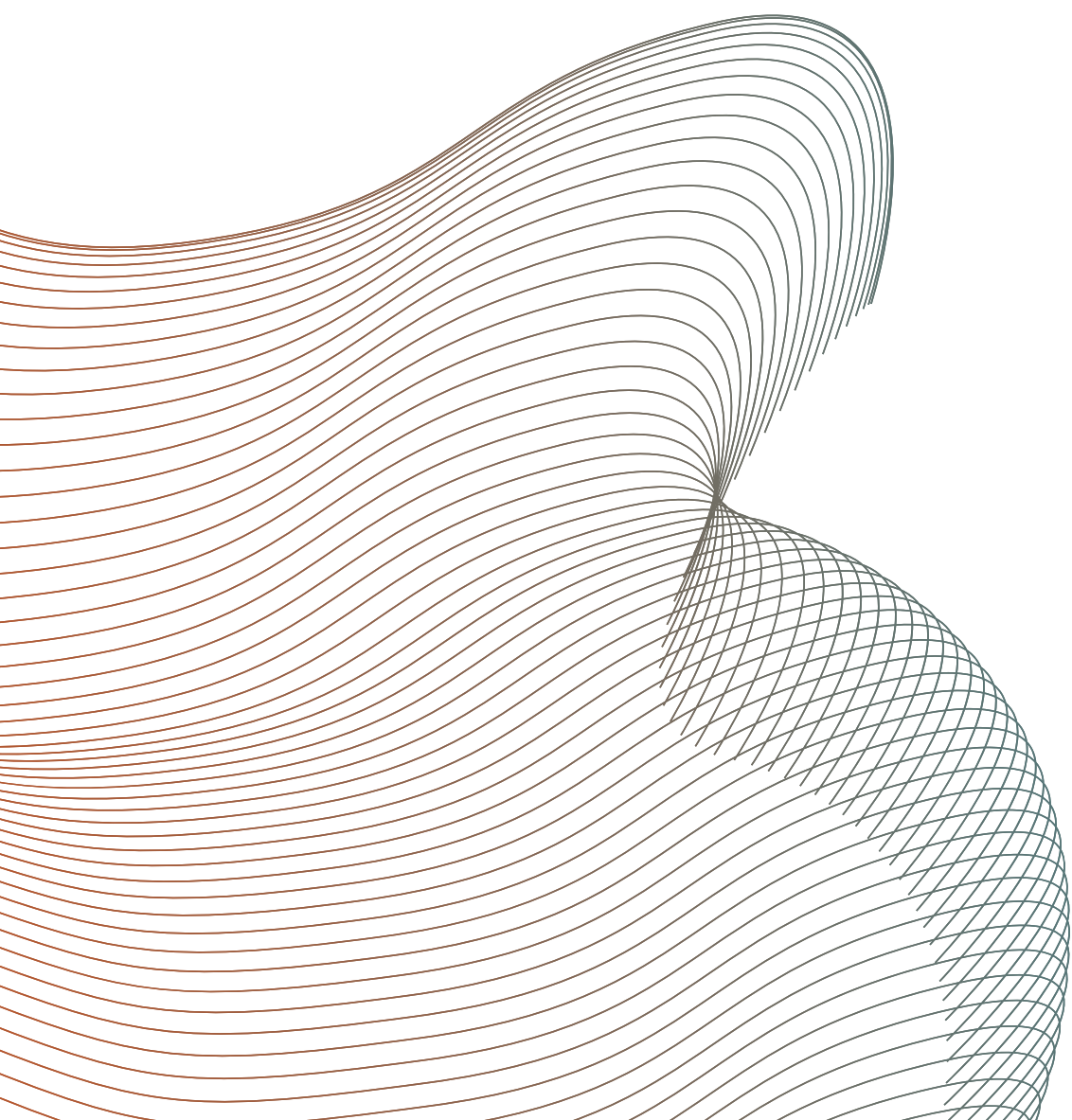
# Install dependencies using uv into the system python
RUN uv pip install --system --no-cache fastapi uvicorn google-cloud-aiplatform spotipy python-dotenv pandas

# Copy the rest of the source code
COPY . .

# Cloud Run uses the PORT env variable
EXPOSE 8080

# The CMD needs to point to the file where 'app = FastAPI()' lives
# use the API path we built :
CMD ["sh", "-c", "uvicorn src.api.main:app --host 0.0.0.0 --port ${PORT:-8080}"]
```





Q & A